Bash and HPC Essentials

1. Set up Aliases for Bash File (.bash_profile)

First set up aliases to log-in to chpc. Begin by determining what SHELL you are using.

```
echo $PATH
```

If you're using a `zsh` shell, modify your `.zshrc` file.

If you're using a `bash` shell, modify your `.bashrc` file.

```
cd
ls -a
vim .zshrc
```

Example of aliases added to rc file:

```
export PATH="/opt/homebrew/bin:$PATH"

alias np='ssh -Y [uid]@notchpeak.chpc.utah.edu'
alias kp='ssh -Y [uid]@kingspeak.chpc.utah.edu'
```

When you re-open your terminal, these settings will be loaded. You can either close and re-open the terminal. Or, you can call:

```
source .zshrc
```

or

```
source .bashrc
```

Now, we'll add aliases when logging into chpc.

```
np
cd
vim .bash_profile
```

Example of added aliases

```
# .bash_profile

# Get the aliases and functions
if [ -f ~/.bashrc ]; then
        . ~/.bashrc
fi

alias watch="watch "
alias sq="squeue --me"
alias sq2="squeue -o \"%8i %12j %4t %10u %20q %20a %10g %20P %10Q %5D %11l %11L %R\""
alias sqNode="squeue --me --sort=\"N\""
alias cl="rm e_* o_* R_* my*"
alias si="sinfo -o \"%20P %5D %14F %8z %10m %10d %11l %25f %N\""
alias siMem="sinfo -o \"%20P %5D %6t %8z %10m %10d %11l %25f %N\" -S \"-m\""
alias siNode="sinfo -o \"%20P %5D %6t %8z %10m %10d %11l %25f %N\" -S \"-N\""
sa() {
  sacct -j "$1" --format=JobID,partition,state,time,elapsed,MaxRss,MaxVMSize,nodelist -X
}

# User specific environment and startup programs

PATH=$PATH:$HOME/bin

export PATH
unset USERNAME

# Restore modules

# module restore clusterMods
```

2. Run a slurm script and transfer results to local machine

We'll simulate estimating pi using chpc, using the script here Applied HPC with R.

First, download the R script file (01-sapply.R) and save to a directory of interest.

On my machine, I've created a new folder `chpc-examples` and saved the `01-sapply.R` file here:

```
cd ~/Library/CloudStorage/Box-Box/__teaching/advanced\ computing/2024
mkdir chpc-examples
cd chpc-examples
curl -O "https://book-hpc.ggvy.cl/01-sapply.R"
ls
```

Generally, I develop the code on my local machine, so I don't need to download the file. Google search for how to download a file using terminal (here) .

Second, set up where you'd like to save the file within your chpc account.

```
cd
mkdir examples
cd examples
mkdir pi
cd pi
pwd
```

Third, securely transfer the file from your local machine to your chpc account in the desired location.

```
# scp [source file(s)] [target location]
scp * -r [uid]@notchpeak2.chpc.utah.edu:~/chpc-examples/pi
```

Fourth, create a slurm file, here we'll follow along submitting-jobs-to-slurm. Save the slurm file as `01-sapply.slurm` and then run calling `$ sbatch 01-sapply.slurm`.

Run the code and note that it needs additional information – specifically the account and partition.

Use the following modification of submitting-jobs-to-slurm:

```
#!/bin/sh
#SBATCH --job-name=sapply
#SBATCH --time=00:10:00
#SBATCH --account=phs7045
#SBATCH --partition=notchpeak-shared-freecycle
#SBATCH --output o_%j
#SBATCH --error e_%j
module load R
Rscript --vanilla 01-sapply.R
```

Rerun with `$ sbatch 01-sapply.slurm` and watch with `watch sq`.

Call `ls -l` and see the new files: `e_[job]`, `o_[job]`, and `01-sapply.rds`.

Optional, take a look at the error and output files.

```
vim e_[job]
```

```
vim o_[job]
```

Optional, take a quick look at output (but don't take more than 10-15 minutes or it'll use up resources on the log-in node).

```
$ module load R
$ R
```

```
o <- readRDS("01-sapply.rds)
mean(o)
```

Fifth, transfer the results back to your local machine. Go back to your local machine and the directory of interest.

```
scp [uid]@notchpeak2.chpc.utah.edu:~/chpc-examples/pi/02-mclpply.rds .
```

SBATCH options

Now, let's practice updating slurm options.

First, clean up directory

```
rm o_* e_*
```

```
cl
```

Optional, copy the slurm file to a file. Notice that the slurm file does not need to end with `.slurm` suffix.

```
cp 01-sapply.slurm 01a-sapply.sl
vim 01a-sapply.sl
```

```
#!/bin/sh
#SBATCH --job-name=sapply
#SBATCH --time=00:10:00
# SBATCH --account=phs7045
#SBATCH --account=owner-guest
# SBATCH --partition=notchpeak-shared-freecycle
#SBATCH --partition=kingspeak-shared-guest
#SBATCH --output out_%j
#SBATCH --error error_%j
#SBATCH --mail-type=START,END,FAIL
#SBATCH --mail-user=jonathan.chipman@hci.utah.edu
module load R
Rscript --vanilla 01-sapply.R
```

Try `sbatch 01a-sapply.sl` and see that it doesn't run. You get the error:

```
sbatch: error: invalid partition specified: kingspeak-shared-guest
sbatch: error: Batch job submission failed: Invalid partition name specified
```

This is because we are not logged onto kingspeak. Now, call:

```
sbatch -M kingspeak 01a-sapply.sl
watch sq -M kingspeak
```

Keep your eye out for an email when it starts and finishes.

Parallel (an exercise)

Now, we'll use the Rscript for using the parallel package Case 2: Single job, multicore job:

Exercise:

1. Pull the file `02-mclapply.R` to your local machine and then send it to hpc.
2. Copy a version of the slurm script to have the name `02-mclapply.slurm` (or a similar name) and add the SBATCH option:

```
#SBATCH --cpus-per-task=4
```

3. Run the results and send it back to your local machine.